Let’s check if it works. First, the Polar UI, Keycloak, Redis, and PostgreSQL containers should be started. Open a Terminal window, navigate to the folder where you keep your Docker Compose configuration (polar-deployment/docker) and run the following:

```
$ docker-compose up -d polar-ui polar-keycloak polar-redis polar-postgres
```

Then run both Edge Service and Catalog Service (`./gradlew bootRun` from each project). Finally, open a browser window, and go to `http://localhost:9000`.

Ensure you can see the list of books in the catalog without being authenticated, but not add, update, or delete them. Then log in as Isabelle (`isabelle`/`password`). She is an employee of the bookshop, so she should be allowed to modify the books in the catalog. Next, log in as Bjorn (`bjorn`/`password`). He is a customer, so he shouldn’t be able to change anything in the catalog.

Under the hood, the Angular application fetches the user roles from the `/user` endpoint exposed by Edge Service and uses them to block pieces of functionality. That improves the user experience, but it’s not secure. The actual endpoints exposed by Catalog Service don’t take roles into account. We need to enforce role-based authorization. That’s the topic of the next section.

12.2.2 Role-based access control with Spring Security and JWT

So far, when talking about authorization, we referred to granting an OAuth2 Client (Edge Service) access to an OAuth2 Resource Server (like Catalog